

Classificazione Prezzi Smartphone

Gruppo di lavoro

Andrea Fortunato, MAT. 670775, a.fortunato42@studenti.uniba.it

[Link al progetto](#)

A.A. 2025-2026

Introduzione

L'obiettivo del progetto è quello di applicare alcune delle conoscenze maturate durante le lezioni di Ingegneria della Conoscenza, applicando algoritmi visti in forma teorica.

In particolare, per questo progetto ho usato un dataset presente sul sito Kaggle chiamato "[Mobile Price Classification](#)", diviso a priori in un dataset di training e uno di test nella quale è completamente assente la feature da predire.

Avendo a disposizione un training set con i dati dei telefoni cellulare e il loro price range con dominio $[0,3]$, l'obiettivo è prevedere il price range dei telefoni presenti nel dataset di test.

Argomenti trattati

[Descrizione del dataset](#)

[Ragionamento logico](#)

[Apprendimento Non Supervisionato](#)

[Apprendimento Supervisionato](#)

[Scelta degli Iperparametri](#)

[Valutazione delle prestazioni](#)

[Curve di apprendimento e Analisi dei risultati](#)

[Conclusioni](#)

Descrizione del dataset

Il dataset utilizzato è formato dalle seguenti features:

- battery_power: capacità della batteria misurata mAh
- blue: variabile booleana che indica se il telefono ha la funzionalità Bluetooth
- clock_speed: velocità del microprocessore nell'eseguire istruzioni
- dual_sim: variabile booleana che indica se il telefono può utilizzare due sim contemporaneamente
- fc: mega pixels della fotocamera frontale
- four_g: variabile booleana che indica se il telefono ha la funzionalità 4G
- int_memory: memoria interna del telefono in Gigabytes
- m_dep: spessore del telefono in centimetri
- mobile_wt: peso del telefono
- n_cores: numero delle unità di elaborazione
- pc: mega pixels della fotocamera principale
- px_height: altezza della risoluzione in pixel
- ram: Megabytes di memoria RAM
- sc_h: altezza dello schermo in centimetri
- sc_w: larghezza dello schermo in centimetri
- talk_time: durata di una intera carica di batteria in chiamata
- three_g: variabile booleana che indica se il telefono ha la funzionalità 3G
- touch_screen: variabile booleana che indica se il telefono ha il touch screen o meno
- wifi: variabile booleana che indica se il telefono ha il wifi o meno
- price_range: feature obiettivo che indica il range del costo del telefono cellulare (0=basso costo, 1=medio costo, 2=alto costo, 3=altissimo costo)

Ragionamento logico

In questo progetto il programma crea automaticamente tramite la libreria `pyswip` la Knowledge Base partendo dal dataset di train. In particolare, per ogni telefono crea un fatto contenente tutte le informazioni su quel telefono, così schematizzato:

```
% telefono(battery_power, blue, clock_speed, dual_sim, fc, four_g, int_memory,
m_dep, mobile_wt, n_cores, pc, px_height, px_width, ram, sc_h, sc_w, talk_time,
three_g, touch_screen, wifi, price_range)
```

Altri fatti sono i range dei possibili valori per ogni caratteristica in ciascuna fascia di prezzo (escluse le variabili booleane che avranno valori pari a 0 o 1). Per esempio, la caratteristica “ram” ha i seguenti fatti:

```
range_ram(0, 256, 1974).
```

```
range_ram(1, 387, 2811).
```

```
range_ram(2, 1185, 3916).
```

```
range_ram(3, 2259, 3998).
```

Quindi questo significa che per un telefono di fascia 0, i valori di ram andranno da 256 a 1974 e così via.

Questi fatti insieme alle seguenti regole vengono utilizzati per creare degli esempi sintetici di telefoni.

```
:- use_module(library(random)).
```

```
genera_intero(Min, Max, X) :- random(Min, Max, X).
```

```
genera_decimale(Min, Max, Out) :- random(Min, Max, X), Out is round(X * 10) / 10.0.
```

Le regole precedenti servono per generare numeri casuali interi e decimali nel range Min e Max. Per i numeri decimali nel caso del dataset preso in esame è interessante solo la prima cifra decimale e per questo arrotondo.

Un'altra regola utile alla creazione di telefoni con caratteristiche realistiche è la seguente:

```
vincolo_connettivita(FourG, ThreeG) :- (FourG == 1 -> ThreeG = 1 ; random(0, 2, ThreeG)).
```

Questa regola permette alla Knowledge Base di stabilire che se un telefono ha la tecnologia 4G, in automatico avrà anche la tecnologia 3G. Ossia se FourG equivale ad 1, allora anche ThreeG dovrà essere necessariamente 1, altrimenti il valore di ThreeG sarà 0 o 1.

Infine, utilizzando la regola “*genera_nuovo_telefono*(PR, telefono(...)) :- ...” presente in forma completa sul file `knowledge_base.pl`, passando al posto di PR il `price_range` desiderato, la query restituirà un nuovo telefono di quella fascia di prezzo.

Ecco alcuni esempi:

```
(Fascia 0): {'Telefono': 'telefono(1438, 0, 1.2, 0, 18, 0, 59, 0.9, 126, 5, 14, 451, 995, 1519, 12, 9, 11, 1, 1, 1, 0)'}  
(Fascia 1): {'Telefono': 'telefono(560, 0, 1.8, 1, 14, 0, 43, 0.5, 160, 1, 18, 1422, 756, 1767, 17, 16, 13, 0, 1, 0, 1)'}  
(Fascia 2): {'Telefono': 'telefono(1485, 1, 1.9, 0, 3, 1, 3, 0.4, 132, 5, 0, 237, 1623, 1192, 10, 11, 7, 1, 1, 1, 2)'}  
(Fascia 3): {'Telefono': 'telefono(1154, 0, 3.0, 1, 14, 1, 52, 0.6, 134, 1, 8, 847, 630, 2577, 9, 14, 3, 1, 1, 1, 3)'}  

```

Quindi, utilizzando una query sulla regola “genera_nuovo_telefono” è possibile creare un dataset sintetico per effettuare per esempio over-sampling o per utilizzare tecniche di validazione che funzionano meglio su un numero di dati maggiore.

Apprendimento non supervisionato

Avendo a disposizione un dataset supervisionato, ossia con i valori di `price_range` già assegnati ai dati, ho deciso di utilizzare l'algoritmo di apprendimento non supervisionato KMeans per verificare la presenza di sottoclassi all'interno di ogni classe di prezzo.

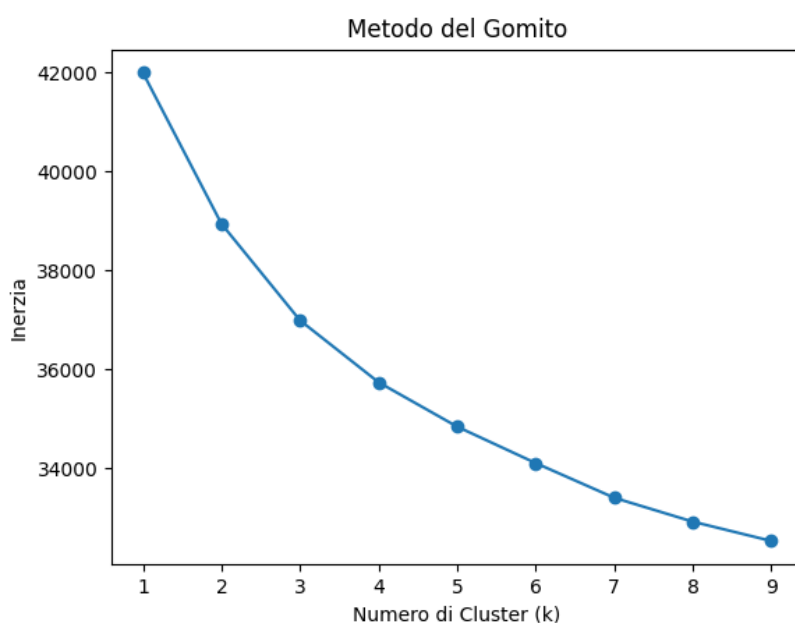
L'algoritmo K-Means è un algoritmo di Hard Clustering, ossia costruisce una funzione che ad ogni esempio assegna un cluster.

Per utilizzarlo ho considerato la feature-obiettivo come variabile latente (non osservando i suoi valori) e diviso il dataset in quattro parti, ognuna delle quali rappresenta una classe di prezzo (0, 1, 2, 3).

Per utilizzare questo algoritmo è necessario trovare il numero ideale di cluster in cui è suddiviso il dataset (l'iperparametro k).

Per determinare il numero reale di cluster, ho deciso di usare il "Metodo del Gomito": il programma disegna su un grafico il variare dell'inerzia all'aumentare del numero di cluster su cui si effettua il K-Means. L'inerzia è la somma dei quadrati delle distanze tra ogni punto del dataset e il centroide del cluster a cui appartiene e nel grafico è indicata sull'asse delle Y. Il numero di cluster è indicato sull'asse delle X.

Per tutte le quattro sezioni del dataset ho ottenuto il valore 4. Ecco un esempio di grafico ottenuto:



Quindi il gomito del grafico indica che 4 è il numero ideale di cluster per il mio dataset. In particolare, in Python ho utilizzato la classe `KneeLocator` della libreria `kneed` inizializzata con il range su cui testare, i valori d'inerzia, il parametro `"curve"` che per il valore `'convex'` rileva il gomito e il parametro `"direction"` che indica la direzione della curva. Inizializzando la classe con questi valori, l'attributo `"elbow"` assume il valore 4 (come è visibile nel grafico precedente).

```
inertia = []
maxK = 10
for i in range(1, maxK):
    km = KMeans(n_clusters=i, n_init=5, init='random')
    km.fit(x_scaled)
    inertia.append(km.inertia_)

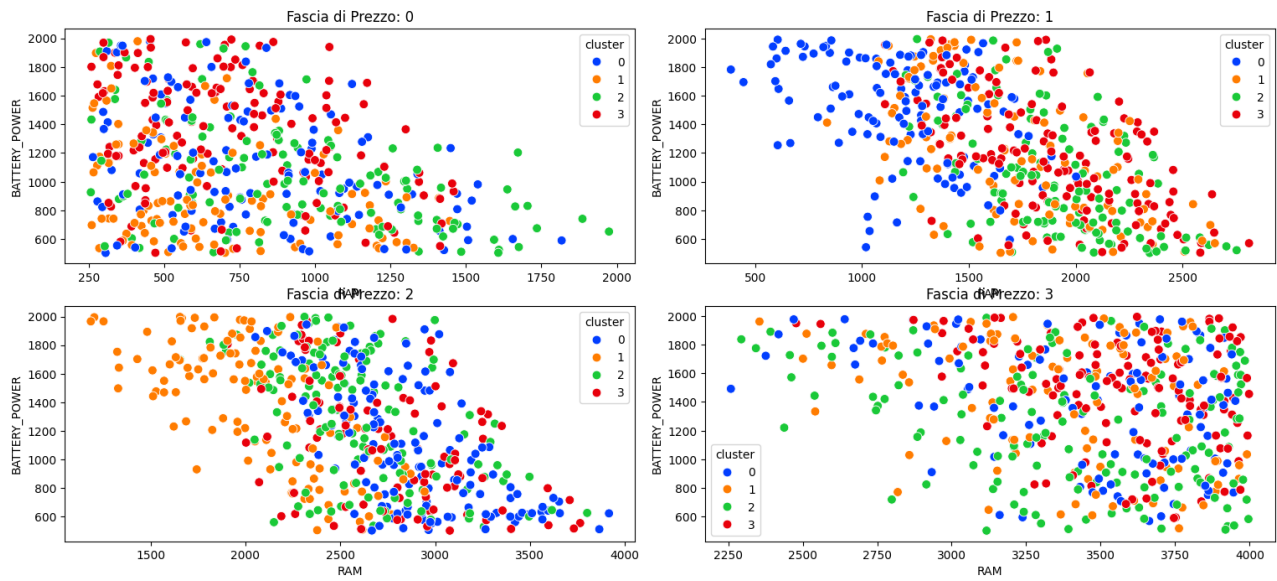
from kneed import KneeLocator
kl = KneeLocator(range(1, maxK), inertia, curve='convex', direction='decreasing')
k_ideale = kl.elbow
```

Dato che all'interno del dataset ci sono dati con unità di misura diverse, ho deciso di scalare i dati utilizzando la classe `StandardScaler` della libreria `sklearn`, per minimizzare la distanza euclidea.

La scelta di utilizzare il metodo di clustering K-Means per vedere la distribuzione dei dati all'interno delle fasce di prezzo, si è rivelata abbastanza fallimentare poiché come evidenza il grafico che segue, i cluster sono molto omogenei poiché i punti sono molto vicini tra loro. Questo significa che per ogni fascia di prezzo avremo telefonini con caratteristiche molto simili tra loro.

Sebbene il clustering sia stato effettuato tenendo conto di tutte le variabili all'interno del dataset, per visualizzare in modo bidimensionale ho scelto di proiettare sugli assi `'ram'` e `'battery_power'` poiché sono le variabili più rappresentative.

Analisi Cluster per Fascia di Prezzo (Basata su RAM e Batteria)



Apprendimento Supervisionato

L'apprendimento supervisionato è una branca dell'apprendimento automatico, il cui obiettivo principale è far sì che l'algoritmo impari una relazione tra i dati di input e output in modo tale da poter fare previsioni accurate su dati mai visti prima.

Nel mio caso l'apprendimento supervisionato si occupa di un problema di classificazione, ossia predire un valore per l'attributo `price_range` compreso nell'insieme finito prefissato {0, 1, 2, 3}.

Per fare ciò ho utilizzato e confrontato tra loro tre modelli di apprendimento supervisionato: Decision Tree, Random Forest e Logistic Regression.

Decision Tree

Il Decision Tree è un albero binario costituito da nodi interni etichettati con le condizioni sulle feature di input e nodi foglia etichettati con una stima puntuale per la feature obiettivo. Quindi per un dato esempio di feature di input, partendo dalla radice e valutando ogni condizione incontrata nei nodi interni si arriverà ad un nodo foglia contenente la stima puntuale.

Random Forest

La Random Forest è un modello che utilizza un numero definito di alberi di decisione addestrati su parti differenti del dataset di training. Per ogni esempio da classificare, ogni albero darà una predizione, le quali saranno aggregate per formare la predizione finale per l'esempio preso in analisi.

Logistic Regression

La Logistic Regression è un modello che cerca di prevedere la probabilità che un dato appartenga a una determinata classe. Durante l'addestramento impara quanto "pesano" le feature.

Per price_range = 3, ossia la classe piu' alta dei prezzi, i pesi di alcune features sono i seguenti:

FEATURE	COEFFICIENT
ram	10.858849
battery_power	2.757012
px_width	1.584425
px_height	1.579760
int_memory	0.248160
fc	-0.014161
clock_speed	-0.045522
m_dep	-0.099580
wifi	-0.102515
mobile_wt	-0.540191

Quindi il modello considera i valori di ram, battery_power, px_width, px_height molto importanti per telefoni costosi; invece, le feature sottostanti saranno presenti sia in telefoni piu' economici che in quelli costosi (dato che sono vicini allo 0).

Scelta degli Iperparametri

Gli iperparametri sono i parametri che vengono fissati prima che il modello possa cominciare l'addestramento. La loro scelta influisce sulle prestazioni e sulla complessità del modello.

Per ogni modello di apprendimento ho eseguito una fase preliminare per la scelta degli iperparametri, ossia tuning tramite cross-validation.

In particolare, ho utilizzato la tecnica di k-Fold Cross Validation. Questa consiste nel dividere il dataset in k fold o insiemi disgiunti e addestrare k volte il modello. Per ogni iterazione 1 fold viene usato per il testing mentre gli altri k-1 fold vengono utilizzati per il training.

Per la ricerca degli iperparametri dei modelli ho usato la strategia GridSearch, ossia il test di tutte le possibili combinazioni di iper-parametri al fine di trovare quella che massimizza le prestazioni dei modelli.

Iperparametri dei modelli

Decision Tree

-criterion: misura la qualità dello split effettuato sui nodi. I possibili valori sono: “gini”, “entropy”, “log_loss”;

gini -> misura quanto spesso un elemento viene classificato in modo sbagliato

entropy -> misura la quantità di disordine nei dati

log_loss -> indica quando l’output corrisponde ad una probabilità piuttosto che a un valore di classe.

-max_depth: indica l’altezza massima dell’albero;

-min_samples_split: indica il numero minimo di esempi richiesti per effettuare lo split sui nodi interni;

-min_samples_leaf: indica il numero minimo di esempi per poter creare una foglia.

Random Forest

Sono gli stessi iperparametri del Decision Tree, con l’aggiunta di n_estimators, il quale indica il numero di alberi nella foresta.

Logistic Regression

-C: indica quanto è forte la regolarizzazione; valori bassi rendono il modello piu’ rigido e prevengono l’overfitting, il contrario per valori alti;

-solver: indica l’algoritmo da utilizzare per apprendere i pesi. I possibili valori sono: “lbfgs”, “liblinear”. Dato che liblinear non supporta classificazioni multiclasse, verrà preso in considerazione solo lbfgs;

-l1_ratio: controlla il mix tra la regolarizzazione L1 ed L2. Dato che lbfgs supporta solo L2, verrà assegnato il valore 0 (l1_ratio=1 -> L1; l1_ratio=0 -> L2);

-max_iter: indica il numero di iterazioni utilizzate.

Assegnazione dei valori agli iperparametri

Come detto, per la scelta degli iperparametri ho utilizzato la tecnica k-fold Cross Validation con $k = 5$ e strategia GridSearch.

I parametri scelti dal programma sono i seguenti:

MODELLO	IPERPARAMETRO	VALORE
Decision Tree	criterion	gini
	max_depth	10
	min_sample_split	2
	min_sample_leaf	10
Random Forest	criterion	entropy
	max_depth	20
	min_sample_split	10
	min_sample_leaf	1
	n_estimators	50
Logistic Regression	C	100
	solver	lbfgs
	l1_ratio	0
	max_iter	10000

Valutazione delle prestazioni

Per valutare i modelli ho utilizzato la k-Fold Cross Validation che addestra il modello diverse volte (nel mio caso 5) utilizzando ogni volta sezioni diverse di esempi di training e test all'interno del Training set.

In particolare, per valutare le prestazioni ho deciso di utilizzare le seguenti metriche: accuracy, precision_macro, recall_macro e f1_macro.

accuracy rappresenta la frazione di previsioni corrette rispetto al totale delle previsioni.

precision_macro è una metrica che calcola la precisione per ogni classe ($price_range = 0, 1, 2, 3$), ossia il numero di esempi classificati in una classe C sul numero di istanze della classe C (precision), e ne calcola la media.

recall_macro è una metrica che per ogni classe calcola il numero di istanze di classe C classificate nella classe C sul numero di istanze della classe C (recall), e ne calcola la media.

f1_macro calcola per ogni classe la media armonica tra precision e recall (f1) e ne calcola la media.

Una volta inizializzato il modello con gli iperparametri visti in precedenza, ho utilizzato il metodo `cross_val_score` della classe `model_selection` della libreria `sklearn` che, con argomento “`cv=RepeatedKfold(n_splits=5, n_repeats=5)`”, divide il training set in 5 parti e ripete l'intero processo 5 volte. In particolare, il modello viene addestrato su 5-1 fold e testato sul fold rimanente. Questo processo viene ripetuto 5, ogni volta con una divisione diversa del training set.

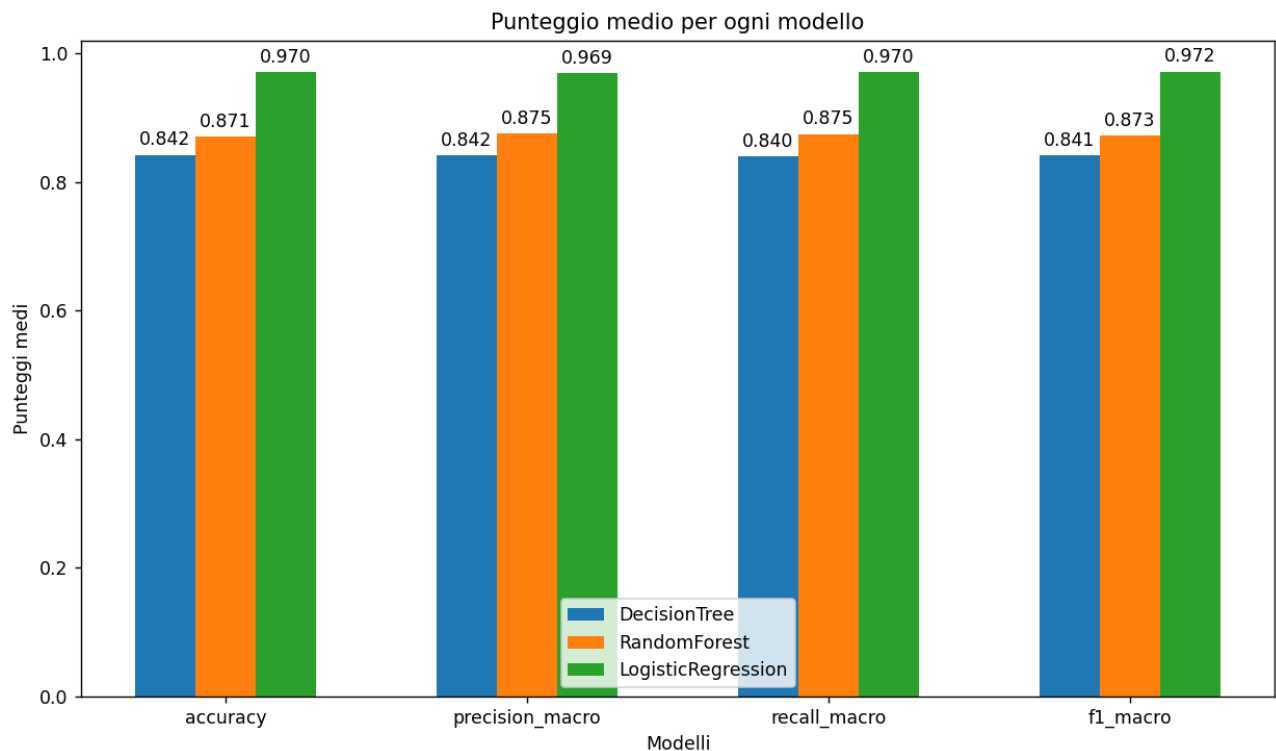
Quindi si avranno (5*5) valori per ogni metrica.

Nella seguente tabella ho inserito la media dei 25 valori per ogni metrica, oltre al valore minimo e massimo.

MODELLO	METRICA	VALORE MEDIO	Min	Max
Decision Tree	Accuracy	0.842	0.797	0.873
	Precision Macro	0.842	0.802	0.882
	Recall Macro	0.840	0.794	0.860
	F1 Macro	0.841	0.803	0.877
Random Forest	Accuracy	0.871	0.843	0.898
	Precision Macro	0.875	0.832	0.895
	Recall Macro	0.875	0.821	0.903
	F1 Macro	0.873	0.819	0.907
Logistic Regression	Accuracy	0.970	0.963	0.977
	Precision Macro	0.969	0.960	0.984
	Recall Macro	0.970	0.957	0.985
	F1 Macro	0.972	0.949	0.982

NB: i valori in tabella sono stati arrotondati alla terza cifra decimale per una piu' facile lettura

Per un piu' immediato confronto tra i modelli, il programma restituisce un grafico a barre con i valori medi:



Questo ci fa capire che dei tre modelli la Logistic Regression sembrerebbe essere il modello piu' affidabile nel fare previsioni su dati mai visti prima. Ma valori delle metriche vicino all'1 potrebbero anche essere sintomo di Sovradattamento (overfitting).

Per Sovradattamento si intende la tendenza dei modelli di apprendimento a imparare correlazioni spurie nei dati che non si riflettono sull'intero dominio del problema. Questo comporta la possibilità che il modello non sarà in grado di predire le classi di appartenenza di esempi mai visti, in assenza di queste correlazioni (comportando errori e diminuendo l'affidabilità del modello).

Per scongiurare la presenza di Sovradattamento ho preso in considerazione la varianza, la deviazione standard e le curve di apprendimento.

La Varianza è la media dei quadrati tra ciascun dato e la media.

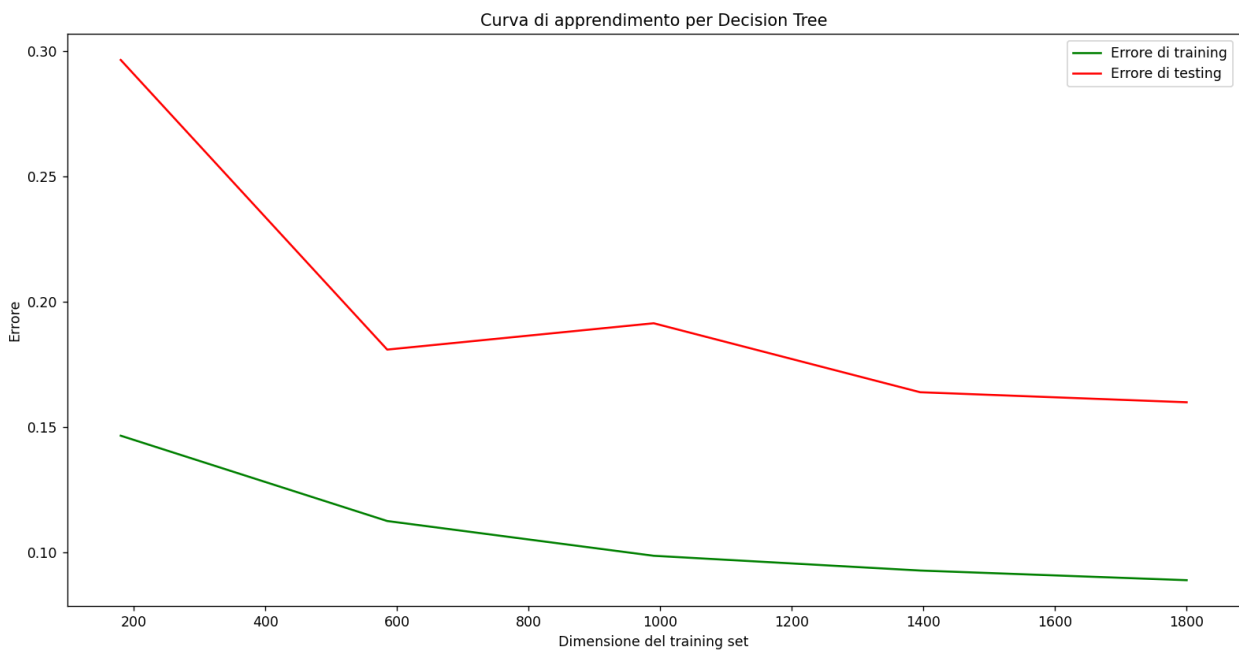
La Deviazione Standard è la radice quadrata della Varianza.

La Varianza indica quanto cambiano le predizioni del modello se questo viene addestrato su un gruppo di dati diverso. Se alta potrebbe essere sintomo di Sovradattamento, ossia il modello potrebbe aver imparato “a memoria” i dati di training e una volta testato con dati nuovi sbaglierebbe nel fare predizioni.

Curve di apprendimento e Analisi dei risultati

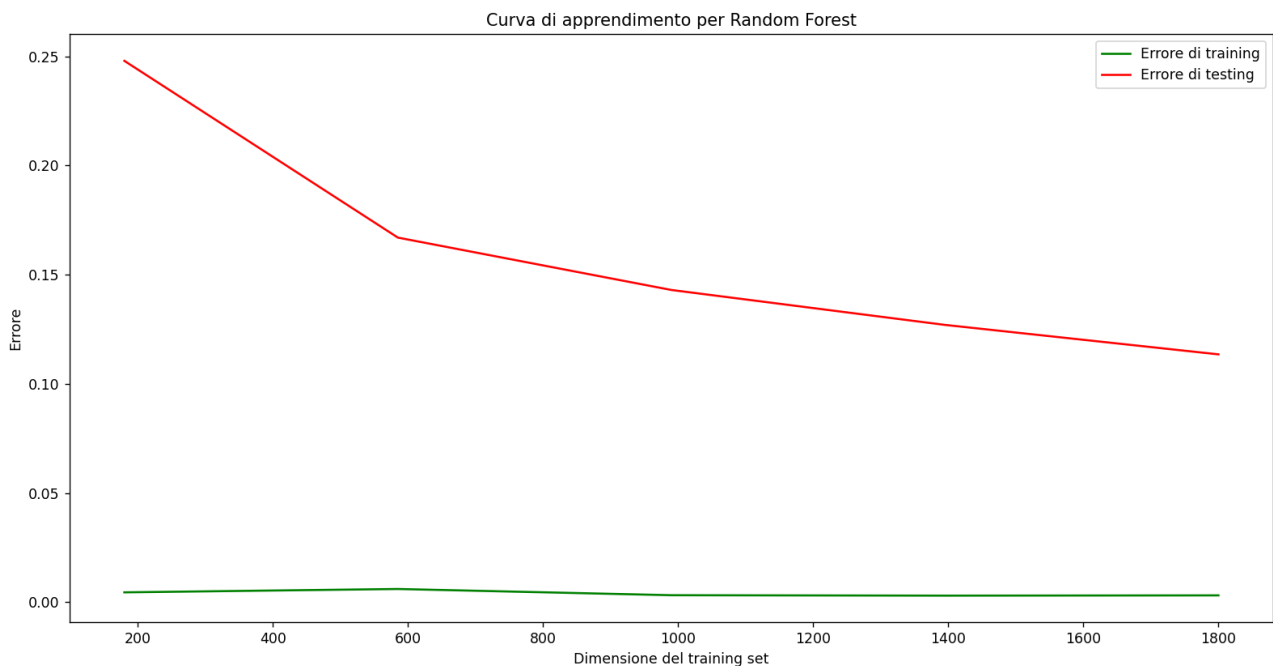
MODELLO	TIPO DI ERRORE	Deviazione Standard	Varianza
Decision Tree	Train	0.002594509873074593	6.731481481481541e-06
	Test	0.012041594578792305	0.00014500000000000024
Random Forest	Train	0.001272937693043318	1.6203703703704453e-06
	Test	0.017036725037400835	0.00029025000000000005
Logistic Regression	Train	0.001410936122133366	1.9907407407407403e-06
	Test	0.010965856099730666	0.00012025000000000024

Decision Tree



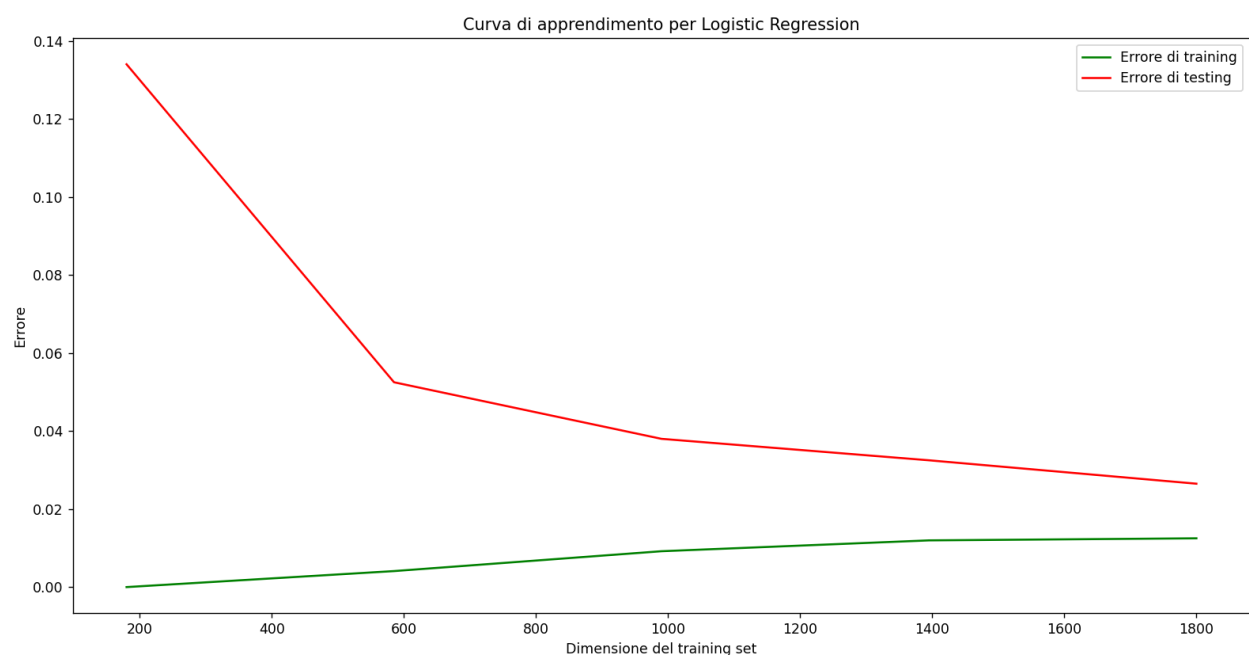
I valori di Deviazione Standard e Varianza sono bassi. Tuttavia, la varianza del test errore è parecchio più alta rispetto a quella del train error (e-06), sintomo probabilmente di overfitting. Inoltre, nella curva di apprendimento è possibile notare un piccolo aumento dell'errore sul test intorno ai 1000 esempi, altro sintomo di overfitting.

Random Forest



Anche nel caso della Random Forest i valori di Deviazione Standard e Varianza sono bassi, con varianza parecchio più alta nel test error rispetto al train error. Inoltre, le curve tendono a rimanere parecchio lontane tra loro. Tutto ciò suggerisce presenza di overfitting.

Logistic Regression



Il grafico mostra che le due curve tendono ad avvicinarsi e i valori di errore ad abbassarsi; quindi, sicuramente il modello risulta migliore rispetto a Decision Tree e Random Forest. Tuttavia, il valore della varianza del test error è parecchio piu' alta rispetto a quella del train error.

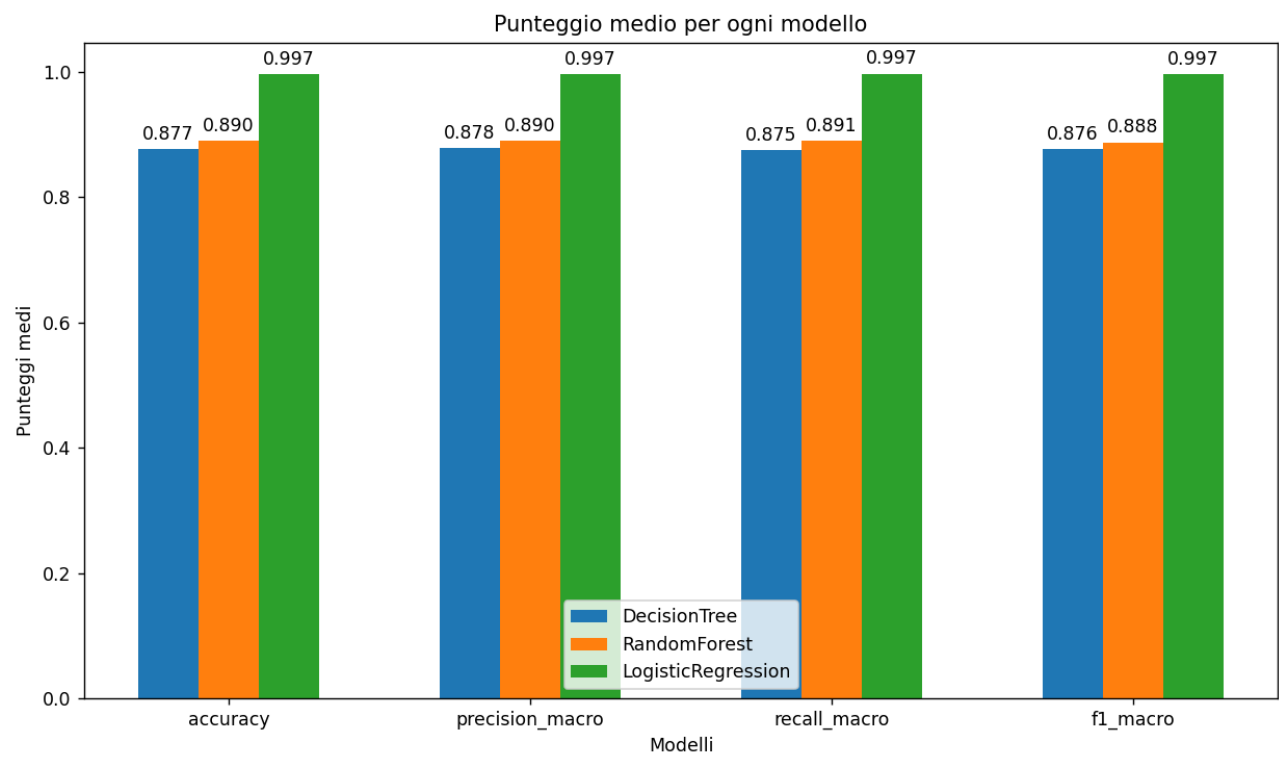
Soluzione all'Overfitting

L'overfitting potrebbe dipendere dalla scarsa dimensione del training set. Per risolvere questo problema ho deciso di effettuare l'oversampling tramite tecnica SMOTE. Solitamente è utilizzata per il bilanciamento delle classi di data set sbilanciati; tuttavia, questa classe della libreria imblearn quando inizializzata con il parametro **sampling_strategy** pari ad un dizionario con chiavi che corrispondono alle classi (nel mio caso {0,1,2,3}) e valore pari ad N, permette di creare N esempi sintetici per ogni classe (nel mio caso N = 5000).

Quindi una volta effettuato l'oversampling il programma ricalcola gli iperparametri:

MODELLO	IPERPARAMETRO	VALORE
Decision Tree	criterion	entropy
	max_depth	5
	min_sample_split	2
	min_sample_leaf	2
Random Forest	criterion	entropy
	max_depth	5
	min_sample_split	10
	min_sample_leaf	1
	n_estimators	50
Logistic Regression	C	100
	solver	lbfgs
	l1_ratio	0
	max_iter	100000

Le prestazioni dei modelli sono leggermente migliorate:



Di seguito possiamo notare come per tutti i modelli i valori di deviazione standard e varianza si siano ridotti, e soprattutto si è ridotta anche la distanza tra le varianze di test e di train.

MODELLO	TIPO DI ERRORE	Deviazione Standard	Varianza
Decision Tree	Train	0.003258546455093144	1.06181250000001e-05
	Test	0.005570008976653444	3.10249999999995e-05
Random Forest	Train	0.007464499313416798	5.571874999999985e-05
	Test	0.005200000000000006	2.7040000000000056e-05
Logistic Regression	Train	0.001741048534648000	3.0312499999999483e-06
	Test	0.003316624790355396	1.0999999999999978e-05

Quindi è possibile affermare che l'oversampling sembra aver risolto l'overfitting.

Conclusioni

Con il presente progetto mi ero posto l'obiettivo di confrontare diversi algoritmi di classificazione per scegliere quello piu' adatto nel predire la classe di prezzo per i telefoni presenti nel file test.csv.

In primis creo tramite ragionamento logico la Knowledge Base contenente i telefoni presenti nel file train.csv e delle regole per la creazione di esempi sintetici di telefoni.

In seguito tramite un'analisi esplorativa con il K-Means è stato possibile confermare che in assenza della feature obiettivo price_range, i dati all'interno delle quattro classi di prezzo sono omogenei.

Per l'obiettivo come era noto già a priori, è piu' adatto usare modelli di apprendimento supervisionato, con task di classificazione.

Quindi si è proceduto alla scelta degli iperparametri tramite K-Fold Cross Validation e GridSearch.

Una volta accertata la presenza di overfitting dovuta alla bassa quantità di dati presenti nel training set, ho deciso di applicare la tecnica SMOTE per effettuare oversampling, il quale sembra aver risolto l'overfitting.

Tra i modelli presi in esame la Logistic Regression è risultato il modello superiore per questo specifico compito. La capacità del modello di dare peso a feature piu' importanti come i valori di "ram" o "battery_power" ha permesso di ottenere un modello equilibrato, con varianza contenuta e un'ottima capacità di generalizzazione.